

Technical Report – Multi-Agent HR Automation Platform

Date: 2026-05-22

Author: Project Maintainer (generated/seeded codebase)

Executive Summary

This document describes the design, implementation, trade-offs, and operational guidance for the Multi-Agent HR Automation Platform. The system demonstrates a compact orchestration pipeline using FastAPI for HTTP, LangGraph for stateful workflow orchestration, lightweight sub-agents for domain handling, a two-tier memory subsystem (STM/LTM) backed by SQLite, and an append-only audit log.

Goals achieved by the system:

- Demonstrable LangGraph orchestration with clearly separated agent boundaries.
- REST API surface for submitting requests and retrieving audit/memory/health status.
- Two-tier memory with heuristic scoring and retrieval logic.
- Append-only audit logging enforced at the DB level.
- LLM integration with safe fallback behavior and no runtime API call at import-time.
- Mocked unit tests enabling repeatable verification without live LLM calls.

System Overview and Architecture

- Entry: `POST /request` – accepts `user_id` and `message`.
- Classifier: keyword-based scoring returns `intent` and `confidence`.
- Memory retrieval: loads recent STM and prioritized LTM per user.
- Router: maps intent to one of the sub-agents (scheduling, leave, compliance, clarification).
- Sub-agent: builds a prompt (injecting STM/LTM) and calls `app.services.llm_service.generate_response`.
- Memory Store: persists STM and conditionally LTM using a heuristic `importance_score`.
- Audit: inserts an append-only record with `created_at` timestamp into `audit_logs` and is enforced via SQLite triggers.

High-level diagram (LangGraph flow):

flowchart LR

```
Request[/POST /request/] --> Classifier[Intent Classifier]
Classifier --> Retriever[Memory Retrieval]
Retriever --> Router[Agent Router]
Router --> SchedulingAgent[Scheduling]
Router --> LeaveAgent[Leave]
Router --> ComplianceAgent[Compliance]
Router --> ClarifyAgent[Clarification]
SchedulingAgent --> MemoryStore[Memory Store]
LeaveAgent --> MemoryStore
ComplianceAgent --> MemoryStore
ClarifyAgent --> MemoryStore
MemoryStore --> Audit[Audit Logger]
Audit --> DB[(SQLite)]
```

Components and Key Files

- `app/main.py` : FastAPI app creation, `init_db()` invocation, and centralized exception handlers for HTTP, validation, and generic errors.
- `app/api/routes.py` : exposes `GET /`, `POST /request`, `GET /audit`, `GET /memory/{user_id}`, `GET /health`, and lists `/docs`.
- `app/agents/orchestrator.py` : constructs the LangGraph `StateGraph`, nodes, edges, and compiles the graph used by the API.
- `app/agents/classifier.py` : lightweight keyword scoring with an explicit low-confidence threshold; returns `intent` and `confidence`.
- `app/agents/router.py` / `app/agents/conditional_edges.py` : maps intents to sub-agents.
- `app/agents/*_agent.py` : sub-agent stubs that construct prompts and call `app/services/llm_service.generate_response`.
- `app/services/llm_service.py` : Groq client wrapper that avoids import-time calls, uses `client` only when `GROQ_API_KEY` is configured, and provides a polite fallback message on failure.
- `app/memory/*` : STM/LTM storage helpers, retrieval, and heuristic scorer in `scorer.py`.
- `app/audit/logger.py` : inserts audit rows; schema enforces append-only with triggers and includes `created_at`.
- `app/core/database.py` : SQLite DB initialization and schema; triggers to block UPDATE/DELETE on `audit_logs`.

- `tests/` : unittest-based tests plus `tests/fixtures/mock_data.json` for deterministic verification.

Data Model (SQLite)

- `short_term_memory` :
 - `id` (INTEGER PK autoinc)
 - `user_id` (TEXT)
 - `message` (TEXT)
 - `response` (TEXT)
- `long_term_memory` :
 - `id` (INTEGER PK autoinc)
 - `user_id` (TEXT)
 - `memory` (TEXT)
 - `importance_score` (INTEGER)
- `audit_logs` (append-only):
 - `id` (INTEGER PK autoinc)
 - `user_id` (TEXT)
 - `request` (TEXT)
 - `intent` (TEXT)
 - `confidence` (REAL)
 - `agent` (TEXT)
 - `response` (TEXT)
 - `status` (TEXT)
 - `created_at` (TEXT DEFAULT CURRENT_TIMESTAMP)

Notes on schema evolution: `init_db()` attempts to add `created_at` if missing, and triggers are created to abort UPDATE/DELETE on `audit_logs` to provide DB-level append-only enforcement.

API Specification

All endpoints return JSON. Key endpoints:

- `GET /` — returns service metadata and endpoint list (includes `/docs`).

- `POST /request` – request body `{"user_id": str, "message": str}`; returns the graph result (includes `response`, `intent`, `confidence`, `memory` fields, and `audit_logged`).
- `GET /audit` – returns audit rows ordered by `created_at DESC, id DESC`.
- `GET /memory/{user_id}` – returns `short_term_memory` (latest 5) and `long_term_memory` (by importance).
- `GET /health` – lightweight health status.

Use the built-in Swagger UI at `/docs` to interactively exercise `POST /request` and inspect responses.

Error Handling & Failure Modes

- Centralized FastAPI exception handlers suppress raw stack traces and return friendly JSON messages for validation (422) and server errors (500).
- The LLM service is guarded: it does not perform remote calls on import and returns polite fallback text when `GROQ_API_KEY` is absent or when the call fails. This prevents startup crashes due to external API outages.

Testing Strategy

- Unit/integration tests are located under `tests/` and use `unittest` with `fastapi.testclient.TestClient`.
- Tests use a small temporary SQLite database and `tests/fixtures/mock_data.json` to seed `short_term_memory`, `long_term_memory`, and `audit_logs` deterministically.
- The `POST /request` path is mocked to avoid exercising LangGraph/LLM internals in CI – this keeps tests fast and deterministic.

Run tests:

```
python -m unittest discover -s tests -p "test_*.py"
```

Security, Privacy, and Compliance

- Avoid logging PII in plaintext in production audit logs; the current demo logs request and response fields as-is for auditability – sanitize or pseudonymize sensitive fields in production.
- Ensure environment variables (LLM API keys) are stored securely (vault, encrypted CI secrets) and not checked into source control.

- SQLite is fine for demos; production should use a managed DB with access control, encryption at rest, and audit capabilities.

Trade-offs and Honest Assessment

This section documents design choices, limitations, and why they are acceptable for a demo/starter codebase.

1. Intent Classification: Keyword-based

- Trade-off: Simple, explainable, and fast but brittle and limited for nuanced user language.
- Rationale: Keeps the pipeline deterministic for early tests and avoids LLM costs. For production, replace with an ML classifier (fine-tuned or prompt-based LLM) that also returns calibrated confidences.

2. Memory Significance Scoring: Heuristic

- Trade-off: Quick to implement and transparent, but not adaptive across users or domains.
- Rationale: Good for a proof-of-concept. Replace with learned ranking or semantic similarity scoring (embeddings + vector DB) for production-quality recall.

3. LLM Integration: Single provider client with fallback

- Trade-off: Uses `groq` client and `GROQ_API_KEY` by default. The service uses a safe fallback when unavailable, but this means degraded responses rather than errors.
- Rationale: Prevents startup failures and offers predictable behavior in offline or test environments. In production, add provider abstraction, circuit breaker, and rate-limit handling.

4. Audit: Append-only enforced by triggers

- Trade-off: SQLite triggers prevent accidental mutation but are limited compared to a secure, immutable audit datastore with cryptographic integrity (e.g., write-once object storage, ledger).
- Rationale: For a small deployment or demo, triggers provide reliable append-only semantics without external infra. For high-integrity needs, use an immutable ledger or sign entries.

5. Tests: Mocked and deterministic

- Trade-off: Tests do not exercise the live LLM or the full LangGraph runtime; they instead verify API wiring, DB interactions, and control flow.
- Rationale: Fast CI feedback and no external dependencies. Add end-to-end staging tests with real LLM and CI secrets guarded for smoke testing.

Operational Guidance

- To reset the DB for a fresh start, stop the app and delete `app.db`.
- Monitor the `app.db` size if LTM or STM grows; consider retention policies or migrating to an external DB/archival store.
- Protect `GROQ_API_KEY` in production – don't place it in source control.

Future Work / Recommendations

- Replace keyword classifier with supervised or prompt-driven classifier that returns calibrated confidence scores.
- Move LTM to vector search (FAISS, Milvus, or hosted vector DB) and store embeddings for semantic retrieval.
- Add RBAC and authentication to API endpoints for production usage.
- Add integration tests that run against a staging environment with a provider LLM key guarded in CI.
- Add metrics, monitoring, and tracing (e.g., Prometheus, OpenTelemetry) to observe end-to-end latencies and failure rates.

Appendix

Sample `POST /request` body

```
{  
  "user_id": "u-100",  
  "message": "Can I take next Friday off?"  
}
```

End of report.